

Introduction to Unix shell

Part 3

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Control Flow and Decision-Making

- The `if` statement allows you to execute commands based on certain conditions

- Basic syntax:

```
if <condition>; then
    # commands to execute if condition is true
fi
```

- Optional `elif` and `else` clauses can be added for additional conditions or default actions

- `elif <condition>; then`
- `else`

- Conditions in `if` statements are typically evaluated using test commands
- Common test commands:
 - `[ <expression> ]`: Basic test command (POSIX compliant)
 - `[[ <expression> ]]`: Extended test command (Bash-specific, more features)
 - `test <expression>`: Another form of the test command
 - `(( <arithmetic_expression> ))`: Evaluates arithmetic expressions and consider output as response code
 - `(command)`: Executes `command` in a subshell and uses its exit status as the condition

- File tests
 - `-f <file>` is a regular file
 - `-d <directory>` is a directory
 - `-e <file>` file exists
 - `-r <file>` file is readable
 - `-w <file>` file is writable
 - `-x <file>` file is executable
- Examples
 - `[ -f "/etc/passwd" ]; echo $?` prints 0 (/etc/passwd is a regular file)
 - `[[ -r "/etc/shadow-" ]]; echo $?` prints 1 (/etc/shadow- is not readable by regular users)

- String comparisons
 - `<string1> = <string2>` strings are equal
 - `<string1> != <string2>` strings are not equal
 - `<string1> < <string2>` string1 is less than string2 (lexicographically)
 - `<string1> > <string2>` string1 is greater than string2 (lexicographically)
 - `-z <string>` string is zero length
 - `-n <string>` string is not zero length
- Examples
 - `[ "apple" = "apple" ]; echo $?` prints 0
 - `[ "apple banana" > "apple_banana" ]; echo $?` prints 1
 - `[[ -n "" ]]; echo $?` prints 1

- Numeric comparisons
 - `<num1> -eq <num2>` num1 is equal to num2
 - `<num1> -ne <num2>` num1 is not equal to num2
 - `<num1> -lt <num2>` num1 is less than num2
 - `<num1> -le <num2>` num1 is less than or equal to num2
 - `<num1> -gt <num2>` num1 is greater than num2
 - `<num1> -ge <num2>` num1 is greater than or equal to num2
- Examples
 - `[ 5 -eq 5 ]; echo $?` prints 0
 - `[ 5 -ne 5 ]; echo $?` prints 1
 - `[ 3 -gt 7 ]; echo $?` prints 1
 - `[ 7 -ge 7 ]; echo $?` prints 0

- Combining conditions with logical operators
 - `<test1> -a <test2>` true if both tests are true (AND)
 - `<test1> -o <test2>` true if either test is true (OR)
 - `! <test>` true if test is false (NOT)

[] vs. [[]]

- Handling variables: `[ $VAR = "foo" ]`
 - If VAR is empty → Error: unary operator expected
 - If VAR contains spaces → Error: too many arguments
 - No problem with `[[ $VAR = "foo" ]]`
- Glob matching: `[[ $filename == *.sh ]]`
- Using Regexp: `[[ $year =~ ^[0-9]{4}$ ]]`
- Logic operators
 - `&&` instead of `-a`
 - `||` instead of `-o`

Conditional Statements (if + [])

```
#!/bin/bash
if [ $#-ne 1 ] ; then
    echo "Usage: $0 <file_path>" >&2 ; exit 1
fi
F="$1"
if [ -e "$F" ]; then
    if [ -d "$F" ]; then
        FILETYPE="directory"
    elif [ -f "$F" ]; then
        FILETYPE="file"
    fi
    echo -n "$FILETYPE $F exists "
    if [ -r "$F" ]; then
        echo "and is readable"
    else
        echo "and is not readable"
    fi
else
    echo "$F does not exist."
fi
```

- Logical operators are used to combine multiple commands or test expressions
- They use shortcut evaluation
 - Can prevent unnecessary (or dangerous) command executions
- Logical AND (&&): Executes the second command only if the first command succeeds (exit status 0)
 - Example: `command1 && command2`
 - If `command1` succeeds, then `command2` is executed
- Logical OR (||): Executes the second command only if the first command fails (non-zero exit status)
 - Example: `command1 || command2`
 - If `command1` fails, then `command2` is executed

Logical Operators Examples

```
#!/bin/bash
# Using logical AND (&&)
make && make install && echo "Compiled and installed successfully"
# Using logical OR (||)
cat /nonexistent_file || echo "File does not exist."
```

- The `for` loop is used to iterate over a list of items or a sequence of numbers

- Basic syntax for iterating over a list of items:

```
for <variable> in <list>; do  
    # commands to execute for each item  
done
```

- Basic syntax for C-style loops (Bash 4.0 and later):

```
for (( initialization; condition; increment )); do  
    # commands to execute for each iteration  
done
```

- Iterating over files using shell expansion

```
for F in /path/to/directory/* ; do echo $F; done
```
- Caveats
 - Can have problem if filename contains spaces or special characters
 - Must explicitly check if the file is a directory
 - Number of elements can exceed ARG_MAX
- Possible mitigation
 - Use `xargs`
 - Use `while` cycle with pipes (will be seen later)
 - Use `find` command (will be seen later)

- The `xargs` command invokes a command on the items provided on the standard input
- Basic usage: `<command> | xargs [options] <program> [init-args]`
- Common options
 - `-I <replace-str>`: Replace occurrences of `<replace-str>` with the input item
 - `-n <number>`: Use at most `<number>` arguments per invocation
 - `-0`: Input items are terminated by a null character instead of whitespace
 - `-p`: Prompt the user before executing each command
 - `-v`: Print the command before executing it
- Examples
 - `ls *.txt | xargs wc -l`: Count lines in all `.txt` files
 - `ls *.log | xargs -I {} mv {} backup/`:
Move all `.log` files to `backup/` directory
 - `echo "file1 file2 file3" | xargs -n 1 rm`: Remove files one at a time

- Given an array:
`NAMES=("Alice" "Bob" "Charlie" "David")`
- Iterating over array values:
`for NAME in "$NAMES[@]"; do`
 `# $NAME contains the current value`
`done`
- Iterating over array indexes: `for INDEX in "$!NAMES[@]"; do`
 `# $INDEX contains the current index`
 `# ${NAMES[$INDEX]} contains the current value`
`done`
- **NOTE:** iteration can cope with the presence of spaces within the array

- Expression without step
`for i in {1..10} ; do`
 # commands to execute for i=1, i=2, ... i=10
`done`
- Equivalent C-style loop: `for ((i=1; i<=10; i++)) ; do`
- Expression with step
`for i in {1..10..2} ; do`
 # commands to execute for i=1, i=3, ... i=9
`done`
- Equivalent C-style loop: `for ((i=1; i<=10; i+=2)) ; do`
- **NOTE:** Brace expressions generate all the possible values when the expression is encountered. They still suffer from the ARG_MAX limitation

- The `while` loop executes a block of commands as long as a condition is true
- Basic syntax:

```
while <condition>; do
    # commands to execute while condition is true
done
```
- The condition is evaluated before each iteration
- If the condition becomes false, the loop terminates
- Useful for processing input, reading files line by line, or implementing counters
- Infinite loops may occur if the condition never becomes false

while loops - Examples

```
#!/bin/bash
# Counter-based loop
COUNT=1
while [ $COUNT -le 5 ]; do
    echo "Count: $COUNT"
    ((COUNT++))
done
# Reading file line by line
while IFS="" read -r LINE; do
    echo "Line: $LINE"
done < input.txt
# User input loop
while true; do
    read -p "Enter a command (q to quit): " CMD
    [ "$CMD" = "q" ] && break
    echo "You entered: $CMD"
done
```

- The **case** statement is used to handle multiple-choice conditions efficiently
- Basic syntax:

```
case <variable> in
    <pattern1>) <commands> ;;
    <pattern2>) <commands> ;;
    <pattern3>) <commands> ;;
    *) <default commands> ;;
esac
```

- The variable is compared against each pattern
- The first matching pattern executes its associated commands
- The **;;** terminates each case branch
- The ***)** case matches any value (default case)
- **esac** marks the end of the case statement

- Patterns can include
 - Literal strings: `"red"`, `"file.txt"`
 - Wildcards: `*.txt`, `test?`, `[aeiou]`
 - Multiple patterns separated by pipes: `"yes" | "y" | "YES"`
- Pattern matching is performed using globbing (not regular expressions)
- Patterns are matched from top to bottom; first match wins
- The default case `*`) is typically the last

The case statement - Examples

```
#!/bin/bash
read -p "Enter a fruit: " FRUIT
case "$FRUIT" in
    "apple"|"Apple"|"APPLE")
        echo "Selected: Apple (red fruit)"
        ;;
    "banana"|"Banana")
        echo "Selected: Banana (yellow fruit)"
        ;;
    "orange"|"Orange")
        echo "Selected: Orange (orange fruit)"
        ;;
    *)
        echo "Unknown fruit: $FRUIT"
        ;;
esac
```

The case statement - File operations

```
#!/bin/bash
read -p "Choose an action (list/create/delete): " ACTION
case "$ACTION" in
    list|ls)
        ls -lah
        ;;
    create|new)
        read -p "Enter filename: " FILENAME
        touch "$FILENAME"
        echo "File created: $FILENAME"
        ;;
    delete|rm)
        read -p "Enter filename to delete: " FILENAME
        rm "$FILENAME"
        echo "File deleted: $FILENAME"
        ;;
    *)
        echo "Invalid action. Use: list, create, or delete"
        ;;
esac
```

Lab experience

- Write a script that checks the available disk space on the `/home` partition (using `df -h`)
- If the space falls below a certain threshold (e.g., 10%), the script should print a warning message to `stderr`
- Otherwise, it should print a success message to `stdout`

- `du` summarizes disk usage of files and directories
- Basic usage: `du [options] [file_or_directory]`
- If no path is specified, analyzes the current directory
- Common options
 - `-h`: Human-readable format (e.g.: 1K, 234M, 2G)
 - `-s`: Display only total size (summary)
 - `-c`: Display total sum at the end
 - `-a`: Show size of all files, not just directories
 - `-d <n>` or `--max-depth=<n>`: Limit output to `n` directories in depth
 - `-x`: Skip directories on different filesystems
- Note: By default, sizes are in 512-byte blocks

- `du -sh /home`: Total size of home directory in human-readable format
- `du -h /home | sort -h`: Display sizes sorted from smallest to largest
- `du -h --max-depth=1 /home`: Show size of subdirectories in /home
- `du -ah /tmp`: Show size of all files and directories in /tmp
- `du -sh *`: Show total size of each file/directory in current directory
- `du -sh /home/* | sort -rh | head -10`: Top 10 largest directories
- `du -c -h *.txt`: Show size of each .txt file and total

- `df` displays disk space usage of mounted filesystems
- Basic usage: `df [options] [file_or_filesystem]`
- Shows total space, used space, available space, and mount point for each filesystem
- Common options
 - `-h`: Human-readable format (e.g., 1K, 234M, 2G)
 - `-i`: Display inode usage instead of disk space
 - `-T`: Show filesystem type (ext4, btrfs, tmpfs, etc.)
 - `-l`: List only local filesystems (exclude network filesystems)
 - `-x <type>`: Exclude specific filesystem types
 - `-t <type>`: Show only specific filesystem types

- `df -h`: Show all mounted filesystems with human-readable sizes
- `df -h /home`: Show filesystem containing /home directory
- `df -T`: Display filesystem types
- `df -i`: Show inode usage per filesystem
- `df -h | grep /dev`: Show only /dev filesystems
- `df -x tmpfs -x devtmpfs`: Exclude temporary filesystems
- `df -t ext4`: Show only ext4 filesystems

- Find what's consuming disk space:
`du -sh * | sort -rh | head -20`
- Check filesystem capacity:
`df -h` or `df -h /`
- Monitor home directory sizes:
`du -sh /home/* | sort -rh`
- Check inode usage:
`df -i` (useful if disk is full but doesn't show in `df -h`)
- Monitor directory growth over time:
`du -sh /path/to/dir > size.log` (run periodically and compare)

- Syntax: `tr [OPTIONS] SET1 [SET2]`
- Translates, squeezes, and/or deletes characters from standard input, writing to standard output.
- Main Options
 - `-c, --complement`: Use the complement of SET1
 - `-d, --delete`: Delete characters in SET1
 - `-s, --squeeze-repeats`: Squeeze repeated characters listed in SET1
 - `-t, --truncate-set1`: Truncate SET1 to length of SET2
- Examples:
 - Convert to uppercase: `echo "hello" | tr 'a-z' 'A-Z'`
 - Delete digits: `echo "abc123def" | tr -d '0-9'`
 - Squeeze spaces: `echo "a b" | tr -s ' '`

```
#!/bin/bash
THRESHOLD=10
PARTITION="/home"
# Compute usate
USAGE=$(df "$PARTITION" | tr -s ' ' | cut -d' ' -f5 | tr -d '%')
# Compute available space
AVAILABLE=$((100 - USAGE))
if [ "$AVAILABLE" -lt "$THRESHOLD" ]; then
    echo "WARNING: Low disk space on $partition. Only $available% remaining." >&2
    exit 1
else
    echo "SUCCESS: Sufficient disk space on $partition. $available% remaining."
    exit 0
fi
```


- Write a script that takes a file extension (e.g., .txt) as an argument.
- Iterates over all files in the current directory with that extension
- Rename them by adding a prefix (e.g., backup-)
- Skip the renaming if the file already has the prefix.

```
#!/bin/bash
EXTENSION=".${1}"
PREFIX="backup-"
if [ "${EXTENSION}" == "." ]; then
    echo "Usage: $0 <extension>"; exit 1
fi
for FILE in *"${EXTENSION}"; do
    if [[ "${FILE}" == "${PREFIX}"* ]]; then
        echo "Skipping $FILE (already has prefix)"
    else
        mv "${FILE}" "${PREFIX}${FILE}"
        echo "Renamed: ${FILE} -> ${PREFIX}${FILE}"
    fi
done
```

Renaming – using xargs

```
# Define the extension and prefix
EXTENSION=".${1}"
PREFIX="backup-"
if [ "${EXTENSION}" == "." ]; then
    echo "Usage: $0 <extension>"; exit 1
fi
ls *"$EXT" | xargs -I {} /bin/bash -c '
    FILE="{}"
    echo "working on $FILE"
    PREFIX="backup-"
    if [[ "$FILE" != "$PREFIX"* ]]; then
        mv "$FILE" "$PREFIX$FILE"
        echo "Renamed: $FILE -> $PREFIX$FILE"
    else
        echo "Skipping $FILE (already has prefix)"
    fi
'
```

- Add colors to the profile personalization script in such a way that colors are used only if the terminal supports them
- Support for piping the output of the script: if the output is not directed to a `tty`, no coloring occurs

- The `$TERM` environment variable specifies the type of terminal being used
- Used by terminal applications to determine which escape sequences and features are supported
- Allows programs to adapt their output based on terminal capabilities
- Example values: `xterm`, `screen`, `rxvt`, `linux`, `vt100`
- Check your terminal type: `echo $TERM`
- Terminal capabilities are defined in the `terminfo` database (typically in `/etc/terminfo` or `/usr/share/terminfo`)

Terminal Type	Description
<code>xterm</code>	Standard X Window terminal emulator. Widely supported and provides good compatibility
<code>screen</code>	Terminal multiplexer. Used when running <code>screen</code> or <code>tmux</code>
<code>rxvt</code>	Lightweight terminal emulator. Popular alternative to <code>xterm</code>
<code>linux</code>	Linux virtual console (framebuffer). Used when running on physical Linux console
<code>vt100</code>	Classic DEC VT100 terminal. Very basic, limited feature set
<code>vt220</code>	Enhanced version of VT100 with more features
<code>ansi</code>	Generic ANSI-compatible terminal
<code>dumb</code>	Minimal terminal with no special capabilities

- Use `tput [-T <terminal>]` command to check terminal capabilities
 - `tput colors`: Display number of colors supported
 - `tput lines`: Display number of lines in terminal
 - `tput cols`: Display number of columns in terminal
 - `tput bold`: Output bold text control sequence
- Use `infocmp` to display detailed terminal information
- Check if output is directed to a terminal (not a pipe) using `test -t 1`
 - `if [ -t 1 ]; then` (stdout is a terminal)
 - `if [ -t 2 ]; then` (stderr is a terminal)
- Conditionally enable ANSI colors only when writing to a terminal

Conditional coloring example

```
#!/bin/bash
# Enable colors only if output is directed to a terminal
if [ -t 1 ]; then
    RED="\e[31m"
    GREEN="\e[32m"
    RESET="\e[0m"
else
    RED=""
    GREEN=""
    RESET=""
fi
echo -e "${GREEN}Success${RESET}"
echo -e "${RED>Error${RESET}"
```


- Multi-extension organizer
 - Create a script that looks at every file in a directory
 - Each file is moved into a folder named after its extension (e.g., .jpg files go to a jpg/ folder)
 - Counts, for every extension the number of files moved
 - Try using `xargs`
- Permission auditor: take a list of sensitive files and use `xargs` to ensure they are not world-readable
 - Define an array of files based on input
 - For each file in the array (use `xargs`) check if the file is world readable
 - If the file is world-readable print a warning
- Log Rotator
 - Look at all .log files.
 - If a log is older than a certain *threshold* variable, compress it
 - If it is old and larger than 10MB delete it.
 - Hint: use `touch -d` to create a file with the threshold date, compare files with `test`

Bonus tips and tricks

The `tput` command

- The `tput` command queries and uses terminal capabilities defined in the `terminfo` database
- Allows scripts to control cursor movement, text formatting, colors, and other terminal features
- Works across different terminal types, making scripts more portable
- Basic usage: `tput [options] <capability> [parameters]`
- Common uses
 - Move cursor to specific positions
 - Clear the screen or specific areas
 - Set text attributes (bold, underline, reverse video)
 - Query terminal dimensions (lines, columns)
 - Control colors and visual effects
- Returns the capability string or value; non-zero exit status if capability is not supported

Common tput Capabilities

Capability	Description
clear	Clear the entire screen
cup <row> <col>	Move cursor to row and column (0-indexed)
cuu <n>	Move cursor up <n> lines
cud <n>	Move cursor down <n> lines
cuf <n>	Move cursor forward <n> columns
cub <n>	Move cursor backward <n> columns
lines	Number of lines in terminal
cols	Number of columns in terminal
bold	Start bold text
smul	Start underline
rmul	Stop underline
rev	Start reverse video
sgr0	Reset all attributes

Cursor Positioning with `tput`

- The `cup` (cursor position) capability moves the cursor to a specific row and column
- Basic usage: `tput cup <row> <column>`
- Row and column numbers are 0-indexed (top-left corner is 0,0)
- Examples
 - `tput cup 0 0`: Move to top-left corner
 - `tput cup 10 20`: Move to row 10, column 20
 - `tput cup $(($(tput lines)-1)) 0`: Move to last line, first column
- Directional movement
 - `tput cuu 5`: Move up 5 lines
 - `tput cud 3`: Move down 3 lines
 - `tput cuf 10`: Move right 10 columns
 - `tput cub 5`: Move left 5 columns
- Use `tput lines` and `tput cols` to get terminal dimensions

Progress Bar Example

```
#!/bin/bash
# Get terminal size
TOTAL_COLS=$(tput cols)
LAST_ROW=$((tput lines) - 1))
# Read and validate input
read -p "Enter a number (0-100): " VALUE
if ! [[ "$VALUE" =~ ^[0-9]+$ ]] || [ "$VALUE" -lt 0 ] || [ "$VALUE" -gt 100 ]; then
    echo "Invalid input. Please enter a number between 0 and 100."
    exit 1
fi
# Calculate bar length proportional to the value
BAR_LENGTH=$((TOTAL_COLS * VALUE / 100))
# Move cursor to the last line, first column
tput cup "$LAST_ROW" 0
# Print the progress bar
BAR_CHAR="*"
printf "%${BAR_LENGTH}s" | tr ' ' "$BAR_CHAR"
# Reset cursor to next line
echo
```